Confidentiality and Authentication?

What This Lab Is About

This lab is designed to test and strengthen your understanding of authentication vulnerabilities and cryptographic weaknesses introduced in:

- Lecture 3: Authentication Methods
- Lecture 4: Protocols for Secure Communication
- Lecture 5: Securing Web Applications

In Lecture 3, we discussed authentication mechanisms including password storage, hashing, and common attacks like dictionary and offline cracking. Lecture 4 covered protocols for secure communication and key establishment. Lecture 5 focused on web application security challenges.

This lab uses realistic web challenges to demonstrate how poor implementation of authentication leads to compromise. You'll practice identifying vulnerabilities, exploiting weak cryptography, and understanding why common practices fail.

Learning Objectives

By completing this lab, you should be able to:

1. Identify client-side vs server-side validation flaws in web applications.
2. Recognize oversharing risks and predictable password patterns.
3. Detect information leakage through HTTP headers and network traffic.
4. Exploit truncated hash collisions and understand MD5 weaknesses.
5. Perform password cracking using tools like John the Ripper on leaked database hashes.

Challenge 1: Insecure Login

Scenario

A web application checks for admin credentials to log into an account. Passwords are generated dynamically, so the users and passwords are not the same. However, some of the developers have made a crucial error that they forgot about before pushing the final version. It's your task to find that.

Your Task

1. Inspect the contents of the website.
2. Figure out where you can find some hidden information in the files.

Takeaways

- Client-side validation provides no real security.
- Secrets can accidentally be leaked by developers.

Questions to Consider

- What could developers leave accidentally that would cause such a data breach?

Challenge 2: TMI on Social Media

Scenario

A user's social media profile reveals personal details that exactly match their password construction pattern. The password follows a predictable formula based on public posts.

Your Task

1. Analyze the profile page and extract key personal information.
2. Deduce the password construction order from available hints.
3. Construct the password and authenticate to the application.
4. Retrieve the flag from the authenticated session.

Takeaways

- Oversharing enables password reconstruction attacks.
- Predictable password formulas undermine security regardless of complexity.
- Social engineering combines with technical analysis.

Questions to Consider

- How does password policy fail when patterns are public?
- Why are date-based and pet-name patterns particularly vulnerable?
- Connect this to dictionary attacks from Lecture 3.

Challenge 3: Single Sign-On Secret Header

Scenario

An SSO/2FA implementation leaks one-time codes. The code is transmitted in plain text, allowing interception and reuse.

Your Task

1. Submit initial credentials and capture the HTTP response.
2. Extract the leaked SSO code from response headers.
3. Reuse the code to complete authentication as admin.
4. Access the admin panel to retrieve the flag.

Takeaways

- HTTP headers are visible to anyone inspecting traffic.
- One-time codes must use secure channels (not custom headers).
- Network proxies reveal implementation flaws.

Questions to Consider

- Why do custom headers fail as secure 2FA channels?
- How does this violate confidentiality from Lecture 1?
- What protocol from Lecture 4 could prevent header leakage?

Challenge 4: MD5 Truncated-Hash Collision

Scenario

A server validates integrity using only the first 7 hex characters of MD5 hashes. You must find a collision against a secret reference without seeing the full message.

Your Task

1. Connect to the service and note the required prefix and target hash.
2. Write a brute-force script to find colliding suffixes.
3. Submit the colliding input to receive the flag.

Takeaways

- Truncating hashes drastically reduces security (28-bit effective strength).
- MD5 collisions are computationally feasible.
- Custom hash validation ignores cryptographic best practices.

Questions to Consider

- Why does 7 hex chars make brute-force viable?
- How does this demonstrate key space analysis from Lecture 2?
- What modern hash function would resist this attack?

Challenge 5: Cracking with John – Web + SQL + Hashes

Scenario

A web app stores hashed passwords in a MySQL database. Database credentials leak through a misconfigured robots.txt, exposing admin hashes. The list of passwords used for the accounts has also been leaked, however, do you have the time to go through all of them for each user?

Your Task

1. Discover leaked database credentials.
2. Dump the users table and extract the hashes.
3. Crack admin passwords using John the Ripper with provided wordlist.
4. Authenticate to the web app as admin to get the flag.

Takeaways

- Leaked database = game over for weak hashing.
- Date-pattern passwords crack quickly even when salted.

Questions to Consider

- Why does SHA-1 fail despite salting?
- Compare the security of the hash you found to bcrypt/Argon2 from Lecture 3.
- How does database leakage amplify web app risks?

General Security Principles

- Never trust client-side validation or exposed secrets.
- Use proper cryptographic primitives (not custom/truncated hashes).
- Protect all authentication factors through secure channels.
- Store passwords with adaptive, slow hashes (bcrypt, Argon2).
- Assume all overshared info enables targeted attacks.

Tools and Techniques

- Browser Developer Tools (Network/Debugger tabs)
- Intercepting proxies (Burp Suite)
- MySQL client for database dumping
- John the Ripper for password cracking
- Python scripting for hash collision brute-force
- netcat (nc) for TCP services

Flag Format

All flags follow: `MaaSec{<16_hex_chars>_flag}` [file:3]

Ethical Reminder

Conduct all exercises ONLY in the provided lab environment. These skills are for defensive cybersecurity only.